

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 4

Выполнил:

Москалец Данила

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

На основе существующей схемы БД и монолитного проекта спроектировать микросервисную архитектуру с принципом database-per-service, описать взаимодействия, эндпоинты (OpenAPI-фрагменты), примеры запросов/ответов, ошибки и пошаговый план миграции.

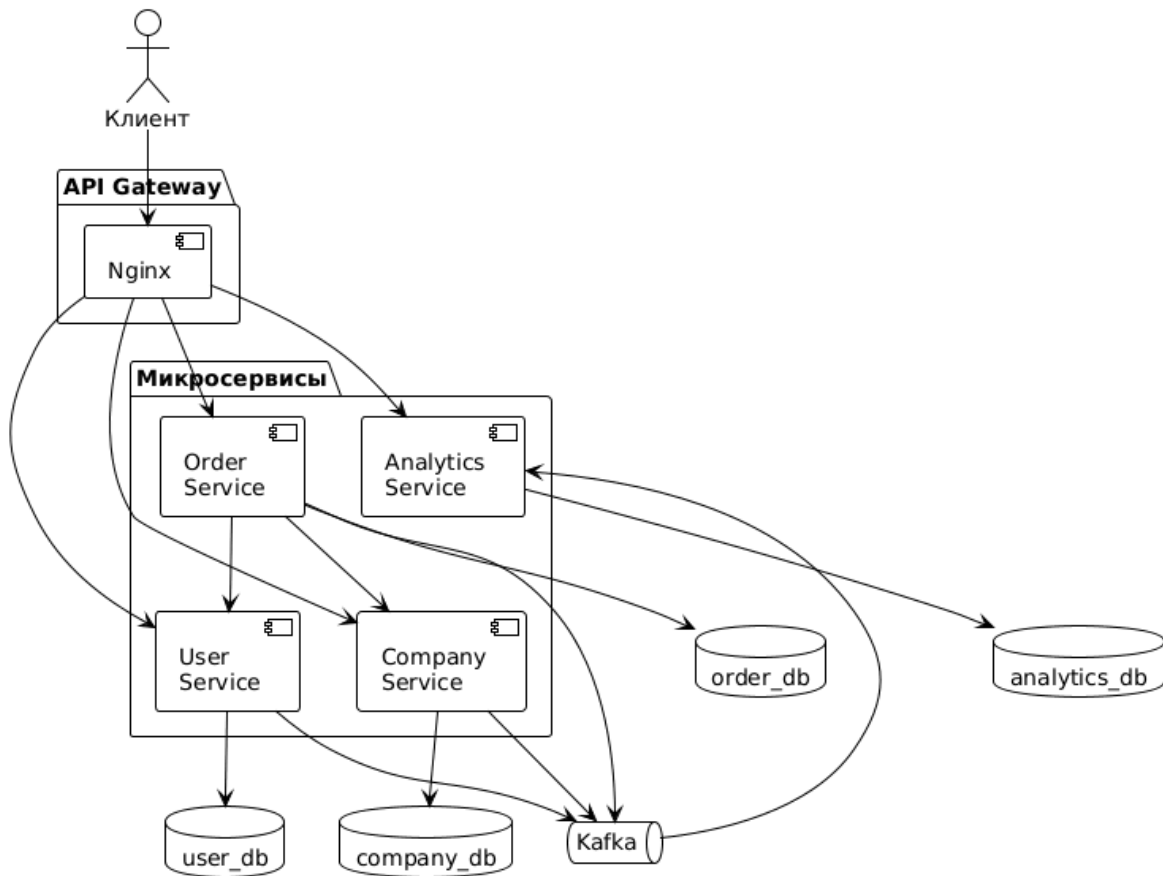
Ход работы

1 Анализ БД и модулей

Исходная схема содержит сущности: User, Company, Category, Service, ServiceCategory, Discount, Request, Review, UserFavorite и набор business-правил (роли, статусы заявок). Для этой лабораторной я предлагаю 4 простых микросервиса:

- User Service: User, аутентификация и профили пользователей;
- Company Service: Company, Service, Category, ServiceCategory, Discount;
- Order Service: Request, Review, UserFavorite;
- Analytics Service: админка и отчеты, агрегированные данные по системе.

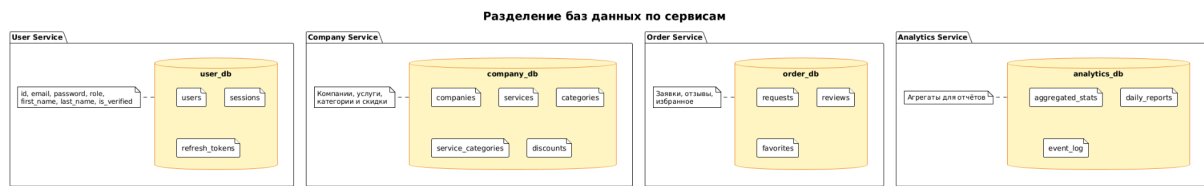
Архитектура микросервисов



2 Database-per-service: предлагаемое разделение БД

- user_db: таблицы *users* (id, email, password, role, is_verified, created_at, updated_at), *sessions*, *refresh_tokens*, *profiles*;
- company_db: таблицы *companies* (id, user_id (owner), title, description, website, logo_url), *services*, *categories*, *service_categories*, *discounts*;
- order_db: *requests* (id, service_id, user_id, status, description, reply, created_at, updated_at), *reviews*, *favorites*;
- analytics_db: *report_views*, *dashboard_metrics*, *event_log* (или материализованные представления для отчётов).

Каждая БД – отдельный Postgres-инстанс / схема, доступный только соответствующему сервису.



db.png

3 Архитектура и схема взаимодействия

- Внешние запросы идут через API Gateway (аутентификация, throttle, TLS, rate-limit, общие CORS). Gateway валидирует JWT через User Service;
- Синхронные межсервисные вызовы (HTTP/REST) применяются для простых проверок и чтения данных;
- Асинхронные события через kafka используются для обмена событиями: `user.updated`, `company.updated`, `request.created`, `review.created`, `favorite.created`;
- Event-driven интеграция: сервис публикует событие, а Analytics Service и другие подписчики обновляют свои данные.

4 Способы взаимодействия, контракты и соглашения

- Gateway валидирует JWT (через Auth) и добавляет `X-User-Id` заголовок для внутренних вызовов;
- Все внутренние API помечать префиксом `/internal` и ограничивать доступ через сеть

5 OpenAPI. Описание эндпоинтов

Для каждого микросервиса разработана отдельная OpenAPI спецификация, описывающая все доступные эндпоинты, форматы запросов и ответов, а также возможные ошибки. Спецификации находятся в файлах:

`user-service.openapi.yaml` – User Service

Auth



POST

/auth/register Регистрация



POST

/auth/login Вход



POST

/auth/refresh Обновить токены



POST

/auth/logout Выход



GET

/auth/me Текущий пользователь



Profile



PUT

/profile Обновить профиль



PUT

/profile/password Сменить пароль



Internal



POST

/internal/validate Валидация JWT



GET

/internal/users/{id} Получить пользователя



company-service.openapi.yaml – Company Service

Companies



GET	/companies	Список компаний	✓
POST	/companies	Создать компанию	🔒 ✓
GET	/companies/{company_id}	Детали компании	✓
PUT	/companies/{company_id}	Обновить компанию	🔒 ✓
DELETE	/companies/{company_id}	Удалить компанию	🔒 ✓

Services



GET	/companies/{company_id}/services	Услуги компании	✓
POST	/companies/{company_id}/services	Создать услугу	🔒 ✓
GET	/services	Каталог услуг	✓
GET	/services/{service_id}	Детали услуги	✓
PUT	/services/{service_id}	Обновить услугу	🔒 ✓
DELETE	/services/{service_id}	Удалить услугу	🔒 ✓

Categories



GET

/categories Список категорий



Discounts



GET

/services/{service_id}/discount Получить скидку



POST

/services/{service_id}/discount Создать скидку



PUT

/services/{service_id}/discount Обновить скидку



DELETE

/services/{service_id}/discount Удалить скидку



Internal



GET

/internal/services/{id} Получить услугу (internal)



order-service.openapi.yaml – Order Service

Requests



POST

/services/{service_id}/requests Создать заявку



GET

/requests/{request_id} Детали заявки



PUT

/requests/{request_id}/status Изменить статус



PUT

/requests/{request_id}/cancel Отменить заявку



Reviews



GET	/services/{service_id}/reviews	Отзывы об услуге	✓
POST	/services/{service_id}/reviews	Оставить отзыв	🔒 ✓
DELETE	/reviews/{review_id}	Удалить отзыв	🔒 ✓

Favorites



POST	/favorites/{service_id}	Добавить в избранное	🔒 ✓
DELETE	/favorites/{service_id}	Удалить из избранного	🔒 ✓

Me



GET	/me/requests	Мои заявки	🔒 ✓
GET	/me/reviews	Мои отзывы	🔒 ✓
GET	/me/favorites	Моё избранное	🔒 ✓

analytics-service.openapi.yaml – Analytics Service

Admin				^
GET	/admin/users	Список пользователей	🔒	✓
GET	/admin/users/{user_id}	Детали пользователя	🔒	✓
DELETE	/admin/users/{user_id}	Удалить пользователя	🔒	✓
GET	/admin/companies	Список компаний	🔒	✓
GET	/admin/requests	Все заявки	🔒	✓
Reports				^
GET	/admin/reports/activity	Отчёт по активности	🔒	✓

6. Примеры ошибок и единый формат

Рекомендуемый формат ошибки (JSON):

```
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "Пользователь не найден",
    "details": null
  }
}
```

Рекомендуемые коды: AUTH_INVALID_TOKEN, AUTH_EXPIRED, USER_NOT_FOUND, SERVICE_NOT_FOUND, VALIDATION_ERROR, CONFLICT, INTERNAL_ERROR.

7. Migrations & Data ownership

- Для каждой сущности определить owner service (см. раздел 1).
Только owner сервис выполняет миграции и имеет право изменять схему своей БД;
- Для миграции данных из монолита применяются простые стратегии;
- Поэтапное переключение маршрутизации: сначала read-only, затем запись, затем полное отключение старой логики.

8. План миграции – пошаговые шаги

- 1) Подготовка инфраструктуры.
 - Настроить user_db, company_db, order_db, analytics_db;
 - Поднять RabbitMQ и API Gateway;
 - Создать репозитории/пакеты для сервисов.
- 2) Вынести User Service.
 - Реализовать auth и профили пользователей;
 - Мигрировать таблицу users в user_db;
 - Настроить Gateway на проверку токенов.
- 3) Вынести Company Service.
 - Мигрировать companies, services, categories, service_categories, discounts в company_db;
 - Перенаправить чтение и запись на новый сервис.
- 4) Вынести Order Service.
 - Создать order_db и сервис, который работает с заявками, отзывами и избранным;
 - Перевести создание заявок, отзывов и избранного на новый сервис.
- 5) Вынести Analytics Service.
 - Подписать сервис на события из RabbitMQ;
 - Хранить агрегаты и показывать отчёты админки.
- 6) Очистка монолита.
 - Перейти на использование только микросервисов;
 - Удалить старую связанную бизнес-логику.
- 7) Тестирование.

Вывод

В ходе выполнения лабораторной работы была спроектирована микросервисная архитектура для маркетплейса охранных услуг на основе существующего монолитного приложения.